

TRANSFORMATION WORKSHOP · 3 DAYS

SCMI Lab Guide

Migrating from Configuration Manager to Microsoft Intune

COURSE

SCMI

EDITION

October 2026

AUTHOR

Michael Happel

Hands-on labs for Day 1 to Day 3 · Lab pod with DC1, SCCM, SRV1, CL1–CL4 and your own tenant

Contents

About this guide

Your lab pod
Accounts
Objects you will meet
Tools

Day 1 – Assess, bridge, connect

Lab 0.1 – Start the identity bridge (30 min)
Lab 1.1 – Assess your site (15 min)
Lab 2.1 – Watch the joins (15 min)
Lab 3.1 – Attach your site (20 min)
Lab 3.2 – Co-manage CL1 (25 min)
Lab 4.1 – Bring CL3 into ConfigMgr (30 min)
Lab 4.2 – Prepare the night (10 min)

Day 2 – Authority, policy, apps

Lab 5.0 – Morning check (10 min)
Lab 5.1 – Who governs? (25 min)
Lab 6.1 – Migrate a GPO (10 min)
Lab 6.2 – Turn a configuration item into a remediation (15 min)
Lab 7.1 – Build a conflict (15 min)
Lab 8.1 – Rebuild three collections (10 min)
Lab 9.1 – Map Greenshot to the Win32 model (15 min)
Lab 10.1 – Convert Firefox ESR with PSADT (35 min)
Lab 10.2 – AI-assisted conversion (30 min)
Lab 10.3 – Repair a required app (optional, 30 min)
Before you leave

Day 3 – Deliver, clean up, operate

Lab 11.0 – Morning check (10 min)
Lab 11.1 – Updates for the pilot ring (25 min)
Lab 12.1 – Device actions (10 min)
Lab 12.2 – CL2 becomes cloud-only (25 min)
Lab 13.1 – Clean sweep (30 min)
Lab 14.1 – Break and fix (25 min)
Lab 15.1 – Your migration plan (35 min)

Appendix

Deliberate dirt – do not fix
Log map

About this guide

Every lab in the SCMI workshop builds on the previous one – work through them in order and do not reset your pod between days. Each lab lists its goal, the steps at task level (not every click) and a checkpoint you confirm before moving on.

- **Steps** say what to do and where. Menu paths use → between levels.
- **Checkpoint** items are the definition of done. If one fails, tell the trainer before you continue – later labs depend on it.
- **Notes** explain what to expect and where it typically goes wrong.

Your lab pod

One isolated pod and one Microsoft 365 E3 or E5 tenant per participant – nothing is shared, co-management settings are site-wide.

System	Role	State at the start
DC1	AD DS and DNS, domain smart.etc	<code>OU=Cloud</code> is the sync scope (User1, User2, CL1, CL2)
SCCM	Primary site ETC, ConfigMgr 2609, Enhanced HTTP	Grown content: apps, collections, baselines, policies, GPOs. Sources in <code>C:\Software</code> , share \SCCM\Software\$
SRV1	Windows Server 2025, domain joined, Sync Server	Operating system only
CL1	Windows 11 26H2, domain joined, ConfigMgr client	<code>OU=Cloud\Workstations\Finance</code> – your pilot device
CL2	Windows 11 26H2, domain joined, ConfigMgr client	<code>OU=Cloud\Workstations\Sales</code> – becomes cloud-only on Day 3
CL3 / CL4	Windows 11 26H2, no domain	In OOB – you join them to Entra ID on Day 1
Tenant	Microsoft 365 E3 or E5	Your tenant admin account from your lab hoster (GoDeploy etc.)

Accounts

Account	Where	Used for
SMART\Administrator	DC1, SCCM	Domain and ConfigMgr administration
User1	Synced from <code>OU=Cloud</code>	Finance user, local administrator – CL1, CL3
User2	Synced from <code>OU=Cloud</code>	Sales user, local administrator – CL2, CL4
Tenant admin	Your tenant	Entra admin center, Intune admin center, Cloud Attach

Passwords are on the pod card from the trainer.

Objects you will meet

- **Collections:** All Workstations, Dept-Finance, Dept-Sales, Pilot-CoMgmt (CL1), Workstations - Production, MW-Workstations Sat 22-02, All Laptops, Finance Laptops, Has 7-Zip installed, Users-App-PuTTY
- **Applications:** 7-Zip 24.09 / 25.01 (supersede), Greenshot (depends on VC++ Redistributable), Notepad++ (two deployment types), PuTTY, Mozilla Firefox ESR, SmartETC Toolbox (deliberately broken), Adobe Reader 2019 (never deployed)
- **Legacy:** package SmartETC Branding, task sequence TS - Finance App Bundle (legacy)
- **Compliance:** BL - Workstation Security, BL - Reporting Only
- **GPOs:** GPO-01 to GPO-04
- **Run Scripts:** Get Uptime, Get Co-Management State, Clear CCM Cache

Tools

- CMTrace or OneTrace for all .log files – ConfigMgr client logs in `C:\Windows\CCM\Logs`, Intune Management Extension logs in `C:\ProgramData\Microsoft\IntuneManagementExtension\Logs`
- Intune admin center: intune.microsoft.com · Entra admin center: entra.microsoft.com
- Your Labfiles are available at <https://tinyurl.com/vxyphyey>

Day 1 – Assess, bridge, connect

By the end of Day 1 your users are synced, CL1/CL2 are hybrid joined, CL3/CL4 are Entra joined, the site is attached to your tenant, CL1 is co-managed and CL3 runs a ConfigMgr client installed from Intune.

Lab 0.1 Start the identity bridge

30 min

Goal: Sync User1, User2 and the computer objects of CL1/CL2 and complete the hybrid join of both devices.

- 1 On DC1, open Active Directory Domains and Trusts and add your tenant's onmicrosoft.com default domain as a new UPN suffix.
- 2 Change the UPNs of User1 and User2 to the new domain suffix.
- 3 On SRV1, download the provisioning agent from Entra admin center → Entra Connect → Cloud sync → Agents and install it. Sign in with your tenant admin and let the wizard create the gMSA with SMART\Administrator.
- 4 Entra admin center → Mobility (MDM and WIP) → Microsoft Intune: set the MDM user scope to All.
- 5 Entra admin center → Cloud sync → New configuration → AD to Microsoft Entra ID sync, domain smart.etc.
- 6 Configuration → Properties → Basics: enable device sync (preview).
- 7 Scoping filter: selected organizational units → `OU=Cloud,DC=smart,DC=etc`
Enable the configuration.
- 8 Create the service connection point (needs Enterprise Admin): on the DC, run `ConfigureSCP.ps1 -Domain <your verified tenant domain> -TenantId <your tenant ID>`. The script is on the Learn page "Configure device sync with Microsoft Entra Cloud Sync" <https://learn.microsoft.com/en-us/entra/identity/hybrid/cloud-sync/device-sync>. It is also in your Scripts folder.
- 9 Assign an M365 E5 or EMS E5 license (depending on your tenant) to the new synced users.
- 10 On CL1 and CL2, sign in and start the scheduled task Automatic-Device-Join (Task Scheduler → Microsoft → Windows → Workplace Join), or run `Start-ScheduledTask -TaskPath '\Microsoft\Windows\Workplace Join\' -TaskName 'Automatic-Device-Join'` in an elevated PowerShell. The join attempt writes the certificate that the sync needs. Then run `dsregcmd /status` on both devices and confirm the hybrid join. User1 and User2 were signed in before the join and have no PRT yet: sign out and in again, wait a minute and check that `AzureAdPrt` is YES.

Checkpoint

- ☐ User1 and User2 are listed in Entra ID as synchronized users
- ☐ After the task has run, the provisioning logs list CL1 and CL2
- ☐ CL1 and CL2: `dsregcmd /status` shows `AzureAdJoined` YES and `DomainJoined` YES
- ☐ After a new sign-in: `AzureAdPrt` YES for User1 on CL1 and User2 on CL2

NOTE

Device sync with Cloud Sync is in preview. The order matters: without a registration attempt by the client, `userCertificate` stays empty and the device does not appear in the provisioning logs at all. Use the task, not `dsregcmd /join` or a restart – in the dry run only the task triggered the sync reliably. The SCP is forest-wide: `CN=62a0ff2e-97b9-4513-943f-0d221bd30080` under `CN=Device Registration Configuration,CN=Services` in the configuration partition.

Lab 1.1 Assess your site

15 min

Goal: A first assessment matrix for the lab site, with evidence for every decision.

- 1 Open the ConfigMgr console on SCCM. Inventory applications and their deployment types, collections, baselines, client settings, packages and task sequences.
- 2 Check deployment status and software metering: what is targeted, what is actually installed and used?
- 3 Use the saved queries (Q - ...) or CMPivot for evidence, for example Q - Installed software (7-Zip).
- 4 Classify at least two objects per category as Migrate, Replace, Redesign, Retire or Retain. Use the sheet 1 Assessment in `SCMI_Migration_Plan_Template.xlsx` or your own notes.
- 5 Write down everything you cannot classify yet.

Checkpoint

- ☐ At least eight objects classified
- ☐ Every Retire entry is backed by evidence (deployment status, metering, empty collection)
- ☐ A list of open questions – they are answered over the next three days

Lab 2.1 Watch the joins

15 min

Goal: Complete the Entra join of CL3/CL4 and compare it with the hybrid join of CL1/CL2 – see what each step left behind.

- 1 Start with CL3 and CL4: complete OOBЕ with a work or school account – User1 on CL3, User2 on CL4.
- 2 Cloud Sync provisioning logs: find CL1 and CL2 – they are listed only after their first join attempt.

- 3 Sign in on CL1 (User1) and CL2 (User2). Both devices are already hybrid joined from Lab 0.1.
- 4 Run `dsregcmd /status` on CL1 and CL3 and compare the Device State and SSO State sections.

Checkpoint

- ☐ CL1/CL2: `AzureAdJoined` YES, `DomainJoined` YES
- ☐ CL3/CL4: `AzureAdJoined` YES, `DomainJoined` NO
- ☐ All four devices in Entra ID with the right join type
- ☐ CL3/CL4 visible in Intune

NOTE

If CL1/CL2 are not joined by the end of the lab, continue with Module 3 – co-management needs CL1 only in the second half of the module.

Lab 3.1 Attach your site

20 min

Goal: Upload your devices to the Intune admin center and run a first action from the cloud.

- 1 ConfigMgr console → Administration → Cloud Services → Cloud Attach → Configure Cloud Attach. Sign in with your tenant admin.
- 2 Upload: upload devices to the Intune admin center, collection All Workstations. On the same tab, clear "Enforce Configuration Manager RBAC for cloud console requests that interact with Configuration Manager".
- 3 Enablement: automatic enrollment in Intune = None (for now).
- 4 Finish the wizard and follow `CMGatewaySyncUploadWorker.log` on SCCM.
- 5 Intune admin center → Tenant administration → Connectors and tokens → Microsoft Endpoint Configuration Manager: open the banner "You can also manage user permissions from Intune", set Use Intune RBAC to On and apply.
- 6 Wait some minutes, then sign out of the Intune admin center and sign in again.
- 7 Intune admin center → Devices → All devices → CL2 → Scripts: run Get Uptime.

Checkpoint

- ☐ CL1 and CL2 visible in Intune, managed by ConfigMgr
- ☐ Script result for CL2 shown in the portal
- ☐ `CMGatewayNotificationWorker.log` shows the action
- ☐ Entra admin center → Devices: the MDM column of CL1 and CL2 still shows None – tenant attach is not an MDM enrollment

NOTE

The Run Scripts must be approved in the console (Software Library → Scripts) before they can run. With Intune RBAC your Global Administrator account is sufficient, but the scripts only appear after a fresh sign-in.

Lab 3.2 Co-manage CL1

25 min

Goal: Enroll CL1 into co-management and move the first workload to Intune for the pilot.

- 1 Cloud Attach properties → Enablement: automatic enrollment = Pilot, pilot collection Pilot-CoMgmt.
- 2 On CL1 as User1, check `dsregcmd /status`: `AzureAdPrt` must be YES (Lab 0.1). Open Edge once and confirm the single sign-on prompt. Then trigger Machine Policy Retrieval & Evaluation Cycle (Configuration Manager control panel → Actions).
- 3 Follow `CoManagementHandler.log` on CL1 until the MDM enrollment completes.
- 4 Cloud Attach properties → Staging: pilot collection Pilot-CoMgmt for all workloads.
- 5 Workloads: move Compliance policies to Pilot Intune and select the Pilot-CoMgmt Collection under Staging.

Checkpoint

- ☐ CL1 shows Co-managed in Intune – a second CL1 entry "ConfigMgr" is expected until both records are merged
- ☐ CL2 stays ConfigMgr only
- ☐ Entra admin center → Devices: the MDM column of CL1 now names Configuration Manager, CL2 still shows None
- ☐ Compliance policies: Pilot Intune
- ☐ `CoManagementHandler.log` without errors
- ☐ Configuration Manager control panel → General → Co-management capabilities: CL1 = 8199 (8192 base + 4 Resource access + 2 Compliance policies + 1 co-management enabled), CL2 = 8197 (no Compliance bit: CL2 is not in Pilot-CoMgmt)

NOTE

If enrollment does not start, check the MDM user scope, the license of User1 and the hybrid join state of CL1 (PRT). A user who has never confirmed the single sign-on prompt (shown in the EU) causes an error in the user MDM sync – opening the browser once after the hybrid join resolves it. A user who was already signed in while the device joined has no PRT (`AzureAdPrt` NO) until the next sign-in.

Lab 4.1 Bring CL3 into ConfigMgr

30 min

Goal: Install the ConfigMgr client on CL3 from Intune and watch it register without an AD account.

- 1 ConfigMgr console → Administration → Cloud Services → Azure Services → Configure Azure Services → Cloud Management. Sign in with your tenant admin, let the wizard create the server app and the client app, and enable Microsoft Entra user discovery.
- 2 Entra admin center → App registrations → All applications: open the two apps the wizard created. Note the Application (client) ID of the client app and the Application ID URI of the server app (Overview or Expose an API). The tenant ID is on the Entra overview page.
- 3 Copy `ccmsetup.msi` from `<site install dir>\bin\i386` to your admin workstation.
- 4 Create a device group SCMI-ConfigMgr-Client with CL3.
- 5 Export the site's "SMS Issuing" root certificate as a .cer file from the ConfigMgr console (Administration → Overview → Security → Certificates). It is not visible in `certlm.msc` on the site server.
- 6 Intune admin center → Devices → Configuration → Create → Windows 10 and later → Templates → Trusted certificate: upload the .cer file, destination store Computer certificate store – Root, assign to SCMI-ConfigMgr-Client.
- 7 On CL3: sync, then confirm in `certlm.msc` that SMS Issuing is listed under Trusted Root Certification Authorities. Do not continue before it is there.
- 8 Intune admin center → Apps → Windows → Add → Line-of-business app: upload `ccmsetup.msi`, command-line arguments:

Text · one long line wraps here – paste it as a single line

```
CCMSETUPCMD="/NoCRLCheck /mp:https://SCCM.smart.etc SMSITECODE=ETC SMSMP=https://SCCM.smart.etc AADTENANTID=<tenant ID> AADCLIENTAPPID=<client app ID> AADRESOURCEURI=<server app ID URI>"
```

- 9 Assign the app as required to SCMI-ConfigMgr-Client.
- 10 On CL3: follow `C:\Windows\ccmsetup\Logs\ccmsetup.log`, then `ClientIDManagerStartup.log` and `LocationServices.log` in `C:\Windows\CCM\Logs`. The folder `C:\Windows\CCM` only appears once ccmsetup got its token from the management point.
- 11 On SCCM: follow `CCM_STS.log` and `MP_RegistrationManager.log`.
- 12 Restart CL3 after the client is installed. In the ConfigMgr console run Update Membership on All Systems and on the collections you want to check: the lab collections only refresh once a day.
- 13 On CL3 open the Configuration Manager control panel and read Co-management capabilities, then open `C:\Windows\CCM\Logs\CoManagementHandler.log`.

Troubleshooting the client installation:

- MSI error 1639 (invalid command-line argument): the closing quotation mark of `CCMSETUPCMD` is missing or is a typographic quote. Type the quotes by hand. MSI log:
`C:\Windows\System32\config\systemprofile\AppData\Local\mdm\ProductCode.log`.
- A corrected command line can take a while to reach the device. Once the bootstrap MSI is installed, Intune does not run it again: test changed parameters locally with `C:\Windows\ccmsetup\ccmsetup.exe` and the

same parameters.

- `ccmsetup.log` shows `800B010A` for `https://SCCM.smart.etc/CCM_STS`: the SMS Issuing root certificate is not trusted on the client yet.
- `ccmsetup.log` shows `80092012`: the certificate revocation check failed, `/NoCRLCheck` is missing in the command line.
- In both cases `ccmsetup` ends with `CCM_E_NO_TOKEN_AUTH ( 0x87d00455 )` and retries every 10 minutes.

What you see on CL3 and CL4, and why:

- Co-management capabilities = 2147479807 (`0x7FFF00FF`), not 8197. `CoManagementHandler.log` shows that the site policy arrived (Capabilities 8197) and is ignored: "Workloads are explicitly set by Intune Co-Management Profile so not using ConfigMgr set workloads".
- Reason: a Windows 11 device that enrolled in Intune first and has no co-management settings policy keeps Intune as management authority. Installing the ConfigMgr client as an app does not change that. Intune manages all workloads, whatever the sliders in ConfigMgr say.
- Intune therefore lists all workloads as Intune-managed for CL3 and CL4. The ConfigMgr client still delivers tenant attach actions, inventory and ConfigMgr app deployments.
- CL3 and CL4 are not members of All Workstations: that collection queries the AD OU, and an Entra joined device has none. Software Center stays empty except for user-targeted deployments (PuTTY for members of GG-App-PuTTY).
- The ConfigMgr nodes of CL3 and CL4 in the Intune admin center can take until the next day to appear.

Checkpoint

- ☐ CL3 in the console: Client = Yes, active
- ☐ CL3 is not in All Workstations – and you know why
- ☐ Configuration Manager control panel on CL3 shows site ETC
- ☐ Software Center opens on CL3

Sync CL3 from Settings → Accounts if the app does not arrive.

Lab 4.2 Prepare the night

10 min

Goal: Everything that needs time can converge before tomorrow morning.

- 1 Cloud Attach properties → Upload: switch from All Workstations to all devices.
- 2 Add CL4 to SCMI-ConfigMgr-Client.
- 3 Intune admin center → Apps → Windows → Add → Microsoft Store app (new): search for Company Portal and assign it as required to SCMI-ConfigMgr-Client. This also installs the Intune Management Extension on CL3 and CL4 overnight – Win32 apps, scripts and remediations on Day 2 need it.
- 4 Leave all VMs of your pod running and note the time.

Checkpoint

- ☐ Upload set to all devices
- ☐ CL4 has the client app assigned
- ☐ Tomorrow: CL3 and CL4 appear once in Intune – with ConfigMgr data

Day 2 – Authority, policy, apps

By the end of Day 2 GPO-01 runs as a settings catalog profile on CL1, the first conflict is resolved, and CL4 – on Intune for all workloads since Day 1 – gets Greenshot, Firefox ESR and Notepad++ from Intune.

Lab 5.0 Morning check

10 min

Goal: Check what converged overnight before you start with Module 5.

- 1 Intune admin center → Devices → All devices: count the entries for CL3 and CL4 – one or two?
- 2 Open CL3: is there a Configuration Manager section in the device menu?
- 3 ConfigMgr console: CL4 with Client = Yes, active?

Checkpoint

- ☐ CL3 has one device record in Intune, with ConfigMgr data – or you have noted the duplicate for Lab 5.1
- ☐ CL4 runs the ConfigMgr client
- ☐ CL3 and CL4: the service Microsoft Intune Management Extension is running and the Company Portal is installed

Lab 5.1 Who governs?

25 min

Goal: Read the workload authority of all four clients, explain why CL1/CL2 follow the site and CL3/CL4 follow Intune – and build the policy that would change it, without assigning it.

- 1 On CL1 to CL4: Configuration Manager control panel → General → Co-management capabilities. Note the value per client and decode it with the workload table from the slides.
- 2 On all four clients read `ConfigInfo` under `HKLM\SOFTWARE\Microsoft\DeviceManageabilityCSP\Provider\MS DM Server`. 1 = Intune is the management authority, 2 = Configuration Manager, no value = the device is not enrolled in Intune.
- 3 On CL3 open `CoManagementHandler.log`. Find the value the site sends (Merged value for setting 'CoManagementSettings_Capabilities') and the value the client applies (Workloads flag retrieved). Read the line that explains the difference.
- 4 Compare with `CoManagementHandler.log` on CL1: here the value from the site is applied.
- 5 On CL3, signed in as User1, open Software Center and install PuTTY. Intune is the authority for every workload on this device, including Client apps – and the ConfigMgr deployment still installs. Authority decides who delivers policies and Intune apps, it does not switch off ConfigMgr deployments.

- 6 Intune admin center → Devices → Enrollment → Windows → Co-management Settings → Create: name SCMI-CoMgmt-Settings. Walk through the settings as if you were going to use it: automatically install the Configuration Manager client, the command-line arguments (your parameters from Lab 4.1) and, under Advanced, "Override co-management policy and use Intune for all workloads".
- 7 Save the policy without an assignment. Do not assign it to any group.
- 8 Decide for each setting combination which `ConfigInfo` value a new device would end up with, and compare with the slide "Who's in Charge?".

Checkpoint

- ☐ CL1: `ConfigInfo` 2, capabilities 8199
- ☐ CL2: no `ConfigInfo` value, capabilities 8197
- ☐ CL3 and CL4: `ConfigInfo` 1, capabilities 2147479807
- ☐ You can explain the value of each device and who decides its workloads
- ☐ PuTTY is installed on CL3 from Software Center, although Intune is the management authority
- ☐ SCMI-CoMgmt-Settings exists and has no assignment
- ☐ The table "Who governs which device" matches your pod

NOTE

CL3 and CL4 enrolled in Intune first, so Intune is their authority and the workload sliders of the site have no effect on them. The co-management settings policy installs the client and sets the authority only during the Autopilot enrollment status page, and Microsoft documents a cloud management gateway as a requirement. Changing it for devices that are already provisioned is described as non-deterministic – that is why the policy stays unassigned in this lab. Setting `ConfigInfo` to 2 by hand is possible, but unusual in production.

Lab 6.1 Migrate a GPO

10 min

Goal: Analyze the lab GPOs and move GPO-01 to Intune for a device where Intune is in charge.

- 1 GPMC on the DC: save GPO-01, GPO-02 and GPO-03 as XML reports (right-click → Save Report..., type XML).
- 2 Intune admin center → Devices → Manage devices → Group Policy analytics → Import the three files.
- 3 Compare the MDM support per GPO and open the per-setting view of GPO-02.
- 4 Select the supported settings of GPO-01 → Migrate → create a settings catalog profile SCMI-GPO-01.
- 5 Cloud Attach properties → Workloads: Device configuration = Pilot Intune.
- 6 Assign SCMI-GPO-01 to a new device group SCMI-CL1-Group with CL1, sync CL1 and check the values under `HKLM\SOFTWARE\Microsoft\PolicyManager\current\device`.

Checkpoint

- ☐ GPO-01 settings arrive on CL1 from Intune
- ☐ You can name three settings of GPO-02 that do not migrate
- ☐ Device configuration on CL1: Intune (pilot)

Lab 6.2 Turn a configuration item into a remediation

15 min

Goal: Rebuild CI - RemoteRegistry Disabled as an Intune remediation and run it on CL4.

WHY

CL3 and CL4 never get BL - Workstation Security, because the baseline is deployed to All Workstations. CI - RemoteRegistry Disabled has no settings catalog equivalent – the Intune counterpart of a script CI is a remediation.

- 1 ConfigMgr console → Assets and Compliance → Compliance Settings → Configuration Items → CI - RemoteRegistry Disabled: open the setting and note the discovery script, the compliance rule (value equals Disabled) and the remediation script.
- 2 Map the three parts: the discovery script and the compliance rule together become the detection script (exit 0 = compliant, exit 1 = not compliant), the remediation script stays a remediation script. The evaluation schedule of the baseline (every 4 hours) becomes the schedule of the assignment.
- 3 Prepare CL4 so that there is something to fix: in an elevated PowerShell run `Set-Service RemoteRegistry -StartupType Manual`.
- 4 Intune admin center → Devices → Scripts and remediations → Remediations → Create: name SCMI-RemoteRegistry-Disabled, the two scripts below, run this script using the logged-on credentials = No, run script in 64-bit PowerShell = Yes.

PowerShell

```
# Detection script

$svc = Get-Service -Name RemoteRegistry -ErrorAction SilentlyContinue

if (-not $svc) { Write-Output 'Service not found'; exit 0 }

if ($svc.StartType -eq 'Disabled') { Write-Output 'Compliant: Disabled'; exit 0 }

Write-Output "Not compliant: $($svc.StartType)"; exit 1
```

PowerShell

```
# Remediation script

Stop-Service -Name RemoteRegistry -Force -ErrorAction SilentlyContinue

Set-Service -Name RemoteRegistry -StartupType Disabled

Write-Output 'RemoteRegistry disabled'; exit 0
```

- 5 Assign it to a new device group SCMI-CL4-Group with CL4, schedule hourly.

- 6 Do not wait for the schedule: Intune admin center → Devices → CL4 → Run remediation → SCMI-RemoteRegistry-Disabled.
- 7 On CL4 check `Get-Service RemoteRegistry | Select-Object Name, StartType, Status` and follow `HealthScripts.log` in `C:\ProgramData\Microsoft\IntuneManagementExtension\Logs`.
- 8 Intune admin center → Remediations → SCMI-RemoteRegistry-Disabled → Device status: add the columns for the detection output before and after the remediation.

Checkpoint

- ☐ CL4: RemoteRegistry start type is Disabled again
- ☐ Device status shows "Issue fixed" with the output of both detection runs
- ☐ You can name what replaced the compliance rule and the baseline schedule of the CI

NOTE

Keep SCMI-RemoteRegistry-Disabled – Lab 12.1 runs it again on demand.

Lab 7.1 Build a conflict

15 min

Goal: Create a real Group Policy / Intune conflict on CL1 with the SmartScreen setting from Lab 6.1, and prove in the registry and the event log which side wins.

- 1 On CL1 check that GPO-01 applies (`gpresult /r`) and read `HKLM\SOFTWARE\Policies\Microsoft\Windows\System: EnableSmartScreen = 1` and `ShellSmartScreenLevel = Block` come from the GPO.
- 2 Intune: open SCMI-GPO-01 from Lab 6.1 and set the SmartScreen setting Prevent Override For Files In Shell to Disabled – the opposite of GPO-01. The same setting now arrives from both sides.
- 3 Add Control Policy Conflict → MDM Wins Over GP = enabled to SCMI-GPO-01.
- 4 Run an MDM sync on CL1 and read the registry key again. Intune does not write there: its values are under `HKLM\SOFTWARE\Microsoft\PolicyManager\current\device\SmartScreen` (`EnableSmartScreenInShell`, `PreventOverrideForFilesInShell`). Read both places: `ShellSmartScreenLevel` is gone from the Policies key.
- 5 Open Event Viewer → DeviceManagement-Enterprise-Diagnostics-Provider → Admin and find two events for `ShellSmartScreenLevel`.
- 6 Run `gpupdate /force` and read the key again: the value stays away. The event log shows a new "MDM wins over GP ... the GP setting is blocked" event, this time with operation Set – Group Policy tried to write the value and was blocked.
- 7 Set the SmartScreen setting in SCMI-GPO-01 back to its original value and sync.

Checkpoint

- ☐ You can show where each side writes its value
- ☐ The event log names the blocked GP setting `ShellSmartScreenLevel`: operation Delete after the sync, operation Set after gpupdate, and the value stays away
- ☐ You can explain why `MDMWinsOverGP` does not replace removing the GPO

NOTE

MDM Wins Over GP only covers settings of the Policy CSP, and Microsoft documents a race condition for settings that are configured on both sides without it. Use it as a safety net during the transition. The clean solution is to remove the setting from the GPO once Intune delivers it.

Lab 8.1 Rebuild three collections

10 min

Goal: Translate three lab collections – and decide which one should not be rebuilt at all.

- 1 **Read the source.** ConfigMgr console → Assets and Compliance → Device Collections. For Pilot-CoMgmt, Windows 11 - Build 26300 and Has 7-Zip installed open Properties → Membership Rules and note three things each: the rule type (direct or query), where the data comes from (a named device, hardware inventory, installed software inventory) and the current members.
- 2 **Pilot-CoMgmt → assigned group.** The collection has one direct rule: CL1. Intune admin center → Groups → New group: type Security, name **SCMI-DEV-Ring-Pilot**, membership type Assigned, add the device CL1. A hand-picked pilot stays hand-picked.
- 3 **Windows 11 - Build 26300 filter.** The collection queries BuildNumber from hardware inventory. Intune admin center → Tenant administration → Assignment Filters → Create → Managed devices: name **SCMI-Win11-Build**, platform Windows 10 and later, rule `(device.osVersion -startsWith "10.0.26300")` with the build number from the collection name.
- 4 In the filter, select Preview devices. Compare the list with the members of the collection: which devices are in the collection but not in the preview, and which are in the preview but not in the collection? The collection sees ConfigMgr clients with hardware inventory, the filter sees devices that are enrolled in Intune.
- 5 Open the assignment of any app or profile you created (do not save): a filter is not a target on its own. It is added to a group assignment in Include or Exclude mode and is evaluated when the device checks in.
- 6 **Has 7-Zip installed → no group, no filter.** The collection queries installed software. Neither a dynamic group nor a filter can read installed software. Write down what the collection was for – typically "give the update only to devices that already have the product" – and express that intent in Intune: a requirement rule on the Win32 app (file, registry or script) so the app is only applicable where 7-Zip exists, or supersedence for a version update.
- 7 Summarise the three translations in one line each: collection, rule type, Intune construct, and what is different afterwards.
- 8 Optional: Groups → New group, membership type Dynamic Device, rule `(device.deviceTrustType -eq "ServerAD")`. Which collection of the lab comes closest to this group, and what does it lose compared

with the OU query?

Checkpoint

- ☐ SCMI-DEV-Ring-Pilot contains CL1
- ☐ The filter preview lists the expected devices
- ☐ You can explain why Has 7-Zip installed has no target equivalent
- ☐ You can name a device that is in the collection Windows 11 - Build ... but not in the filter preview, and say why

NOTE

Keep the groups – they are used again in M9, M10 and M11.

Lab 9.1 Map Greenshot to the Win32 model

15 min

Goal: Rebuild Greenshot and its runtime dependency as Win32 apps and deploy them to CL4.

- 1 Download `IntuneWinAppUtil.exe` (GitHub: microsoft/Microsoft-Win32-Content-Prep-Tool).
- 2 Package `C:\Software\VCRedist` (setup file `vc_redist.x64.exe`) and `C:\Software\Greenshot` (`greenshot-installer.exe`). Save the packages under `C:\Software\Intune`.
- 3 ConfigMgr console: open the deployment types of Greenshot and Microsoft Visual C++ Redistributable x64 and note install command, uninstall command and detection method of each.
- 4 Intune admin center → Apps → Windows → Add → Windows app (Win32): create **Greenshot** first. Install command `greenshot-installer.exe /VERYSILENT /NORESTART /SUPPRESSMSGBOXES /ALLUSERS`, uninstall command `"%ProgramFiles%\Greenshot\unins000.exe" /VERYSILENT /NORESTART`, install behavior System. Detection: manually configured, rule type File, path `%ProgramFiles%\Greenshot`, file `Greenshot.exe`, file or folder exists, associated with a 32-bit app on 64-bit clients = No. Do not assign the app yet.
- 5 Create **Microsoft Visual C++ Redistributable x64** as the second Win32 app. Install command `vc_redist.x64.exe /install /quiet /norestart`, uninstall command `vc_redist.x64.exe /uninstall /quiet /norestart`, install behavior System. Do not assign it.
- 6 Detection of the VC++ app – the registry rule is written differently than in ConfigMgr. ConfigMgr stores the hive and the key separately (Local Machine + `SOFTWARE\Microsoft\VisualStudio\14.0\VC\Runtimes\x64`). Intune wants one full key path: `HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\VisualStudio\14.0\VC\Runtimes\x64`, value name `Installed`, detection method Integer comparison, operator Equals, value 1, associated with a 32-bit app on 64-bit clients = No.
- 7 Only now connect the two: Greenshot → Properties → Dependencies → add Microsoft Visual C++ Redistributable x64 with Automatically install = Yes.
- 8 Only then deploy: assign Greenshot as available to the existing device group SCMI-CL4-Group. The VC++ app gets no assignment of its own – it arrives through the dependency.

- 9 Sync CL4, install Greenshot from the Company Portal and follow `AppWorkload.log` in `C:\ProgramData\Microsoft\IntuneManagementExtension\Logs` : the dependency is detected and installed first, then Greenshot.

Checkpoint

- ☐ Greenshot and VC++ installed on CL4
- ☐ Both apps report Installed in Intune
- ☐ You can name what you did not take over from the ConfigMgr app

OPTIONAL

Rebuild SmartETC Toolbox as a Win32 app and take the detection over unchanged (file `toolbox.txt` in `%ProgramData%\SmartETC\Tools`). Assign it to SCMI-CL4-Group: the script installs, Intune reports the app as failed because it is not detected after the installation – the same deliberate error as in ConfigMgr. Correct the detection path to `%ProgramData%\SmartETC\Toolbox` .

Lab 10.1 Convert Firefox ESR with PSADT

35 min

Goal: Replace the legacy Firefox wrapper with a PSADT v4 package and deploy it via Intune.

- 1 On your packaging machine: `Install-Module PSAppDeployToolkit -Scope CurrentUser` , then `New-ADTTemplate -Destination C:\ADTSoftware -Name FirefoxESR` .
- 2 Copy `FirefoxESR.msi` to Files and `policies.json` to SupportFiles (source: `C:\Software\FirefoxESR`).
- 3 Open `C:\Software\FirefoxESR\Install-Firefox.ps1` and `Uninstall-Firefox.ps1` next to the new `Invoke-AppDeployToolkit.ps1` . The legacy install script does five things: stop Firefox, run the MSI, accept exit codes 0 and 3010, copy `policies.json` , remove the desktop shortcut. Each one gets a place in the template.
- 4 **Variables.** In the `$adtSession` block at the top set `AppVendor` , `AppName` , `AppVersion` and `AppArch` , and put Firefox into `AppProcessesToClose` (code below). `AppName` must not stay empty: with an empty `AppName` and a single MSI in Files the toolkit switches to Zero-Config MSI and installs the MSI on its own.

PowerShell

```
# Variables - in the $adtSession block
AppVendor = 'Mozilla'
AppName = 'Firefox ESR'
AppVersion = '140'
AppArch = 'x64'
AppProcessesToClose = @( @{ Name = 'firefox'; Description = 'Mozilla Firefox' } )
```

- 5 **Pre-Install** – nothing to write. The template already calls `Show-ADTInstallationWelcome` with `AppProcessesToClose` . That replaces `Stop-Process` : the user is asked to close Firefox and can defer.
- 6 **Install** – under "<Perform Installation tasks here>" add the `Start-ADTMsiProcess` line (code below). The file name is enough, the toolkit looks in Files. Silent switches, MSI logging and the handling of exit code

3010 come from the toolkit.

PowerShell

```
# Install phase  
Start-ADTMsiProcess -Action Install -FilePath 'FirefoxESR.msi'
```

- 7 **Post-Install** – under "<Perform Post-Installation tasks here>" copy `policies.json` from SupportFiles and remove the desktop shortcut. Delete the `Show-ADTInstallationPrompt` block at the end of the function: a required deployment should not end with a message box.

PowerShell

```
# Post-Install phase  
Copy-ADTFile -Path "$($adtSession.DirSupportFiles)\policies.json" -Destination "$envProgramFiles\Mozilla Firefox\distribution"  
Remove-ADTFile -Path "$envCommonDesktop\Firefox.lnk"
```

- 8 **Uninstall** – in `Uninstall-ADTDeployment`, under "<Perform Uninstallation tasks here>", call the Firefox uninstaller and remove the remaining folder (code below). Pre-Uninstall again needs nothing: the template closes Firefox with a 60 second countdown.

PowerShell

```
# Uninstall phase  
$helper = "$envProgramFiles\Mozilla Firefox\uninstall\helper.exe"  
if (Test-Path -LiteralPath $helper) { Start-ADTProcess -FilePath $helper -ArgumentList '/S' }  
Remove-ADTFolder -Path "$envProgramFiles\Mozilla Firefox"
```

- 9 Compare: which lines of the legacy script have no counterpart in your package, and which behavior is new?
- 10 Test locally before Intune sees the package. Copy the folder FirefoxESR to CL4 and run from an elevated prompt: `Invoke-AppDeployToolkit.exe -DeploymentType Install`, then `-DeploymentType Uninstall`, then Install again. Read the toolkit log and the MSI log in `C:\Windows\Logs\Software` after each run.
- 11 Package with IntuneWinAppUtil: source folder `C:\ADTSoftware\FirefoxESR`, setup file `Invoke-AppDeployToolkit.exe`, output `C:\Software\Intune\FirefoxESR`.
- 12 Create the Win32 app Mozilla Firefox ESR: install command `Invoke-AppDeployToolkit.exe -DeploymentType Install`, uninstall command `Invoke-AppDeployToolkit.exe -DeploymentType Uninstall`, install behavior System. Return codes: add 1602 as Retry (the user deferred). Detection as in ConfigMgr: file `policies.json` in `%ProgramFiles%\Mozilla Firefox\distribution` exists, associated with a 32-bit app on 64-bit clients = No.
- 13 Make the deployment visible: on CL4 delete `%ProgramFiles%\Mozilla Firefox\distribution\policies.json` so that the app is no longer detected, and start Firefox. Then assign the app as required to SCMI-CL4-Group and sync CL4.

Checkpoint

- ☐ Firefox ESR installed by Intune on CL4
- ☐ `about:policies` shows the `policies.json` settings
- ☐ Toolkit log in `C:\Windows\Logs\Software`
- ☐ The close-apps prompt appeared for the user

NOTE

Typical errors: `policies.json` in Files instead of SupportFiles, detection on the wrong path, v3 function names copied from internet examples.

Lab 10.2 AI-assisted conversion

30 min

Goal: Convert Notepad++ – both deployment types – into one PSADT package with an AI assistant, and prove it works.

- 1 Read the source.** ConfigMgr console → Applications → Notepad++ → Deployment Types. There are two: Notepad++ Finance (requirement: global condition SmartETC Department equals Finance, install `Install-Finance.ps1`, detection registry value `Edition = Finance` under `HKLM\SOFTWARE\SmartETC\NotepadPP`) and Notepad++ Standard (install `npp-installer-x64.exe /S`, script detection on `notepad++.exe`). ConfigMgr picks the first deployment type whose requirements are met. A Win32 app has no deployment types – the decision has to move into the package.
- 2 Create the package folder:** `New-ADTTemplate -Destination C:\ADTSoftware -Name NotepadPP`. Copy `npp-installer-x64.exe` to Files.
- 3 Build one input file.** Copilot accepts only a few uploads per prompt, so everything the model needs goes into a single text file. On SCCM, in the ConfigMgr PowerShell session, run the code block below. It writes `C:\ADTSoftware\NppInput\NppInput.txt` with four marked sections: the legacy script, the application XML with both deployment types, the untouched template and the list of valid functions.

PowerShell

```
# Build one input file for the model – run in the ConfigMgr PowerShell session on SCCM

Import-Module PSAppDeployToolkit

$in = 'C:\ADTSoftware\NppInput'

New-Item -Path $in -ItemType Directory -Force | Out-Null

$parts = [ordered]@{

    'LEGACY SCRIPT: Install-Finance.ps1' = Get-Content -Path 'C:\Software\NotepadPP\Install-Finance.ps1' -Raw
    'CONFIGMGR APPLICATION XML (both deployment types)' = (Get-CMApplication -Name 'Notepad++').SDMPackageXML
    'TEMPLATE: Invoke-AppDeployToolkit.ps1' = Get-Content -Path 'C:\ADTSoftware\NotepadPP\Invoke-AppDeployToolkit.ps1' -Raw
    'VALID FUNCTIONS (Get-Command -Module PSAppDeployToolkit)' = (Get-Command -Module PSAppDeployToolkit).Name -join "`r`n"
}

$parts.GetEnumerator() |
    ForEach-Object { "===== $_.Key) =====`r`n$_.Value`r`n" } |
    Set-Content -Path "$in\NppInput.txt" -Encoding UTF8
```

- 4 Open `NppInput.txt` and check it before it leaves your machine: four sections, no credentials, no license keys, no customer names.
- 5 **Prompt.** Open Microsoft 365 Copilot Chat (m365.cloud.microsoft/chat) with your tenant admin, upload `NppInput.txt` and paste the prompt below. Do not shorten the rules – each one closes a typical gap.

Text

```
Role: packaging engineer, PSAppDeployToolkit v4.1+, Intune Win32 apps.

Task: convert the legacy install into the template (both in the attachment).

    The ConfigMgr application has two deployment types (see the XML):

    Finance and Standard. Build ONE package that decides at run time.

Decision:

- Read HKLM\SOFTWARE\SmartETC\Department (64-bit view).
- Value 'Finance' -> Finance edition. Any other value or no value -> Standard.
- Write the decision to the toolkit log.

Rules:

- Use only functions from the section VALID FUNCTIONS.
- Set AppVendor, AppName, AppVersion in $adtSession.
- Close processes via $adtSession.AppProcessesToClose.
- No ServiceUI. No hard-coded user paths.
- Keep every step of the legacy script or explain why not.
- Repair must evaluate the department again.
- Uninstall removes the Edition value.

Output:

1. Invoke-AppDeployToolkit.ps1 (complete)
2. Intune detection rule and return codes
3. Test checklist: install, uninstall, repair, prompt

Attachment: NppInput.txt with four sections marked by ===== lines:

    legacy script, ConfigMgr application XML, template, valid functions
```

- 6 Save the answer as `Invoke-AppDeployToolkit.ps1` in `C:\ADTSoftware\NotepadPP`. Save the prompt, the model name and the date as `Prompt.txt` next to it.
- 7 **Read before you run.** Check by reading: `AppName` is set (otherwise Zero-Config MSI logic applies), the department is read from `HKLM\SOFTWARE\SmartETC\Department`, a missing value leads to Standard, the installer is started from Files with `/S`, only the Finance branch writes Edition, the uninstall phase calls `uninstall.exe /S` and removes the Edition value, and the decision is written to the log.
- 8 **Function check.** Models invent function names that sound right, often from the old v3 toolkit. Open a PowerShell window, change to `C:\ADTSoftware\NotepadPP`, copy the code block below, paste it into the window and press Enter. The block is not part of `Invoke-AppDeployToolkit.ps1` – it is a separate test that you run against the script. The block reads `Invoke-AppDeployToolkit.ps1` without running it, collects every command with `-ADT` in its name and prints those the installed module does not know. No output is the good result: every ADT function in the script exists. If a name is printed, that line of the script would fail at run time – replace the function or give the name back to Copilot. To see the check react, add a line `Remove-ADTNonsense` to the script, run the block again, and remove the line. The check only looks at names, not at parameters.

PowerShell

```
# Function check - lists every ADT command the script uses that the module does not know
Import-Module PSAppDeployToolkit

$ast = [System.Management.Automation.Language.Parser]::ParseFile(
    (Resolve-Path '.\Invoke-AppDeployToolkit.ps1'), [ref]$null, [ref]$null)

$ast.FindAll({ $args[0] -is [System.Management.Automation.Language.CommandAst] }, $true) |
    ForEach-Object { $_.GetCommandName() } | Sort-Object -Unique |
    Where-Object { $_ -like '*-ADT*' -and -not (Get-Command $_ -ErrorAction Ignore) }
```

- 9 **Analyzer.** `Install-Module PSScriptAnalyzer -Scope CurrentUser`, then `Invoke-ScriptAnalyzer -Path .\Invoke-AppDeployToolkit.ps1 -Severity Warning, Error`.
- 10 Fix what the two checks and your reading found – by hand or by giving the finding back to Copilot in the same chat. Write down one thing the model got wrong.
If you need a working ps1, you can find it in the Course Labfiles Folder: the complete script (built for PSAppDeployToolkit 4.1.8) and `NotepadPP-Blocks.txt` with the parts to paste into your own template.
- 11 **Test on CL4.** Copy the folder NotepadPP to CL4 and run from an elevated prompt: `Invoke-AppDeployToolkit.exe -DeploymentType Install`. CL4 has no Department value: expect the Standard edition, no Edition value under `HKLM\SOFTWARE\SmartETC\NotepadPP`, and the decision in the toolkit log in `C:\Windows\Logs\Software`. Then test `-DeploymentType Uninstall`.
- 12 **Package and deploy.** IntuneWinAppUtil: source folder `C:\ADTSoftware\NotepadPP`, setup file `Invoke-AppDeployToolkit.exe`, output `C:\Software\Intune\NotepadPP`. Win32 app Notepad++: install command `Invoke-AppDeployToolkit.exe -DeploymentType Install`, uninstall command `Invoke-AppDeployToolkit.exe -DeploymentType Uninstall`, install behavior System, return code 1602 as Retry.
- 13 **Detection:** one rule that is true for both editions – file `notepad++.exe` in `%ProgramFiles%\Notepad++` exists, associated with a 32-bit app on 64-bit clients = No. The registry rule of the Finance deployment type would report Standard devices as failed.
- 14 Assign the app as required to SCMI-CL4-Group and sync.
- 15 Compare with ConfigMgr: where did the two deployment types, the global condition and the two detection methods go?

Checkpoint

- ☐ The function check returns nothing
- ☐ CL4 has Notepad++ Standard and no Edition value
- ☐ The log shows the department decision
- ☐ You can name one thing the model got wrong

OPTIONAL

On CL4 create the string value `Department = Finance` under `HKLM\SOFTWARE\SmartETC` and run `Invoke-AppDeployToolkit.exe -DeploymentType Install` again from the elevated prompt. Check that the log names the Finance edition and that `Edition = Finance` exists under `HKLM\SOFTWARE\SmartETC\NotepadPP`.

Lab 10.3 Repair a required app

Optional

30 min

Goal: Give the helpdesk a way to force the reinstall of a required app – with a marker in the PSADT template and a remediation on demand.

WHY

Intune has no repair action. For a required app, Reinstall in the Company Portal does nothing while detection is positive, and Uninstall is not passed to the device. The lever that remains is the detection rule.

Part A – Marker in the package

- 1 Open `C:\ADTSoftware\FirefoxESR\Invoke-AppDeployToolkit.ps1` from Lab 10.1. Note the value of `AppName` in the `$adtSession` block – it becomes the name of the marker key: `HKLM\SOFTWARE\SmartETC\Apps\<AppName>`, for example `...\Apps\Firefox ESR`.
- 2 **Write the marker.** In `Install-ADTDeployment`, at the very end of the Post-Install phase (after the `Remove-ADTFile` line), add the first four lines of the code below. They create the key and write Version and InstallDate.

PowerShell

```
# End of the Post-Install phase - company marker (same pattern in every package)

$marker = "HKLM:\SOFTWARE\SmartETC\Apps\$($adtSession.AppName)"

New-Item -Path $marker -Force | Out-Null

Set-ItemProperty -Path $marker -Name 'Version' -Value $adtSession.AppVersion

Set-ItemProperty -Path $marker -Name 'InstallDate' -Value (Get-Date -Format s)


# End of the Post-Uninstallation phase

Remove-Item -Path "HKLM:\SOFTWARE\SmartETC\Apps\$($adtSession.AppName)" -Recurse -Force -ErrorAction SilentlyContinue
```

- 3 **Remove the marker.** In `Uninstall-ADTDeployment`, under "<Perform Post-Uninstallation tasks here>", add the `Remove-Item` line from the same block.
- 4 **Make the Install phase repeatable.** Intune will run the install command again on a device where Firefox is already present. The Firefox MSI does not support a repair through Windows Installer, so the package installs over the existing version. In the Install phase add the switch `-SkipMSIAlreadyInstalledCheck` to the `Start-ADTMsiProcess` line from Lab 10.1 (code below). Without it the toolkit sees that the MSI is already installed and skips the installation without a message.

Text

```
## <Perform Installation tasks here>

Start-ADTMsiProcess -Action Install -FilePath 'FirefoxESR.msi' -SkipMSIAlreadyInstalledCheck
```

- 5 **Test locally.** Copy the folder to CL4 and run `Invoke-AppDeployToolkit.exe -DeploymentType Install` from an elevated prompt. Firefox is already installed, so the toolkit log in `C:\Windows\Logs\Software` must show that the MSI is installed again and not skipped. Check the marker: `Get-ItemProperty 'HKLM:\SOFTWARE\SmartETC\Apps\<AppName>'` shows Version and InstallDate.

- 6 **Repackage.** IntuneWinAppUtil: source folder `C:\ADTSoftware\FirefoxESR`, setup file `Invoke-AppDeployToolkit.exe`, output `C:\Software\Intune\FirefoxESR`.
- 7 **Update the app.** Intune admin center → Apps → Windows → Mozilla Firefox ESR → Properties → App information → Edit → App package file: select the new .intunewin. Commands and assignment stay as they are.
- 8 **Change the detection.** Properties → Detection rules → Edit. Remove the `policies.json` rule and add two rules – both must be true: (1) rule type Registry, key path `HKEY_LOCAL_MACHINE\SOFTWARE\SmartETC\Apps\<AppName>`, value name Version, detection method Value exists, associated with a 32-bit app on 64-bit clients = No. (2) rule type File, path `%ProgramFiles%\Mozilla Firefox`, file `firefox.exe`, detection method File or folder exists, 32-bit = No.
- 9 Delete the marker key on CL4 once by hand and sync. Firefox is installed, but the marker is missing: with the new detection the app counts as not installed, Intune runs the install command again and the package writes the marker. This is exactly the lever Part B pulls remotely.

Part B – Remediation on demand

- 1 Note the current InstallDate in the marker key on CL4.
- 2 Intune admin center → Devices → Scripts and remediations → Remediations → Create: name SCMI-Repair-Firefox. Paste the two scripts below and replace `<AppName>` in both with the `AppName` of your package. Run this script using the logged-on credentials = No, enforce script signature check = No, run script in 64-bit PowerShell = Yes.

PowerShell

```
# Detection script - reports an issue as long as the marker exists
$marker = 'HKLM:\SOFTWARE\SmartETC\Apps\<AppName>'
if (Test-Path $marker) { Write-Output 'Marker present'; exit 1 }
Write-Output 'Marker already removed'; exit 0
```

Text · one long line wraps here – paste it as a single line

```
# Remediation script - removes the marker, clears the re-evaluation schedule, forces a full report, restarts the IME
$marker = 'HKLM:\SOFTWARE\SmartETC\Apps\<AppName>'
Remove-Item -Path $marker -Recurse -Force -ErrorAction SilentlyContinue
# Clear the re-evaluation schedule so that the IME runs detection at the next check-in
Remove-Item -Path 'HKLM:\SOFTWARE\Microsoft\IntuneManagementExtension\Win32Apps*\GRS' -Recurse -Force -ErrorAction SilentlyContinue
# Force a full status report so that Intune receives the new state
Remove-ItemProperty -Path 'HKLM:\SOFTWARE\Microsoft\IntuneManagementExtension\Win32Apps\Reporting\*' -Name 'LastFullReportTimeUtc' -Force -ErrorAction SilentlyContinue
# Restart the IME after the result has been reported
Start-Process powershell.exe -WindowStyle Hidden -ArgumentList '-ExecutionPolicy Bypass -Command "Start-Sleep 180; Restart-Service IntuneManagementExtension -Force"'
Write-Output 'Marker removed, schedule cleared, full report forced, IME restart scheduled'; exit 0
```

- 3 On the Assignments page select **no group**. The remediation is only run on demand: assigned to a group it would remove the marker at every scheduled run and reinstall Firefox again and again.
- 4 Intune admin center → Devices → Windows → CL4 → ... → Run remediation → SCMI-Repair-Firefox → Run remediation.

- 5 On CL4 follow `HealthScripts.log` in `C:\ProgramData\Microsoft\IntuneManagementExtension\Logs`: the detection script exits with 1, the remediation script runs, the marker key is gone.
- 6 About three minutes later the Intune Management Extension restarts. Follow `AppWorkload.log`: Mozilla Firefox ESR is evaluated as not detected, the install command runs, the toolkit log shows the MSI installation.
- 7 Read the marker key again: `InstallDate` is newer than the value from step 1.

Checkpoint

- ☐ The marker key exists after the Intune installation, with `Version` and `InstallDate`
- ☐ After the remediation, `AppWorkload.log` shows the app as not detected and installs it again
- ☐ The toolkit log shows that the MSI was installed again, not skipped
- ☐ You can explain why the remediation must not be assigned to a group

NOTE

Replace `<AppName>` with the `AppName` of your package. Assigned to a group, the remediation would remove the marker on every schedule run. The remediation also deletes the GRS keys under `HKLM\SOFTWARE\Microsoft\IntuneManagementExtension\Win32Apps`. They hold the re-evaluation schedule of the Win32 apps; with the keys gone, the Intune Management Extension runs detection again at its next check-in instead of waiting for the schedule. The line clears the schedule for all apps, not only for Firefox: apps with a positive detection are left alone, failed apps are retried earlier than after 24 hours. The restart of the Intune Management Extension brings that check-in forward. Deleting `LastFullReportTimeUtc` under `...\Win32Apps\Reporting` makes the next status report a full one – without it the reinstallation happens on the device, but the Intune admin center shows it only much later. The restart is delayed on purpose: the remediation script itself runs inside the Intune Management Extension. Restarting the service at once would stop the script before its result is reported to Intune, so a separate process waits until the report is sent.

Before you leave

- ☐ Firefox and Notepad++ deployments on CL4 installed
- ☐ All VMs of your pod running

Day 3 – Deliver, clean up, operate

By the end of Day 3 Windows updates for CL1 come from Intune, CL2 is cloud-only, CL4 runs without ConfigMgr, every device has exactly one record – and you leave with a draft migration plan for your own environment.

Lab 11.0 Morning check

10 min

Goal: Check the app deployments of Day 2 before you start with Module 11.

- 1 Intune admin center → Apps → Windows: device install status of Mozilla Firefox ESR and Notepad++ for CL4.
- 2 On CL4: `AppWorkLoad.log` in the IME log folder and the toolkit logs in `C:\Windows\Logs\Software`.
- 3 Which Notepad++ edition did the wrapper choose?

Checkpoint

- ☐ Both apps installed on CL4 – or the failure is traced from IME log to toolkit log to MSI log
- ☐ CL1 to CL4 running and signed in

Lab 11.1 Updates for the pilot ring

25 min

Goal: Hand Windows updates for CL1 to Intune and replace the branch bandwidth rule with Delivery Optimization.

- 1 Intune admin center → Devices → Windows updates → Update rings: create SCMI-Ring-Pilot (quality deferral 0, deadline 2 days, grace period 1 day), assign to SCMI-DEV-Ring-Pilot.
- 2 Feature updates: create a policy SCMI-Anchor for Windows 11, version 26H2, same group.
- 3 Settings catalog profile SCMI-DO: download mode Group (2), group ID source DNS suffix, and the intent of CS - Branch Graz. That client setting limits BITS to 512 Kbps from 08:00 to 17:00 and to 4096 Kbps outside. ConfigMgr counts kilobits, Delivery Optimization kilobytes: 4096 Kbps = 512 KB/s. So set Max Background Download Bandwidth (in KB/s) = 512 and Set Business Hours to Limit Background Download Bandwidth: 8 to 17, 12 % during business hours (12 % of 512 KB/s ≈ 64 KB/s = 512 Kbps), 100 % outside. Same group.
- 4 ConfigMgr: create a custom client setting CS - Pilot-CoMgmt with Software Updates = No and deploy it to Pilot-CoMgmt. The slider only moves the authority for the Windows Update policies – the software update agent of the ConfigMgr client stays active until a client setting switches it off. Microsoft: "After moving the Windows Update workload to Intune, the client settings in Configuration Manager need to be adjusted manually."
- 5 Cloud Attach properties → Workloads: Windows Update policies = Pilot Intune.

- 6 Sync CL1, run Machine Policy Retrieval. Check `HKLM\SOFTWARE\Microsoft\PolicyManager\current\device\Update`, Settings → Windows Update and `Get-DeliveryOptimizationStatus`.

Checkpoint

- ☐ `PolicyManager\current\device\Update` shows the ring values on CL1
- ☐ Settings → Windows Update shows that policies are managed by your organization
- ☐ `CoManagementHandler.log`: Windows Update on Intune

NOTE

Feature update reports can take hours – do not wait for them.

Lab 12.1 Device actions

10 min

Goal: Use the daily actions from Intune on an enrolled device and see what tenant attach offers for a ConfigMgr-only device.

- 1 CL4 in Intune: run the remediation SCMI-RemoteRegistry-Disabled from Lab 6.2 on demand and Collect diagnostics.
- 2 Download the diagnostics zip and find the IME logs in it.
- 3 CL2 in Intune: compare the available actions with CL4 and run the script Get Co-Management State (CL2 → CM Scripts → Run script, as in Lab 3.1). It is one of the ConfigMgr Run Scripts of the lab site and reads `ConfigInfo` under `HKLM\SOFTWARE\Microsoft\DeviceManageabilityCSP\Provider\MS_DM_Server` – the value from Lab 5.1. Expected for CL2: an empty result, because CL2 is not enrolled in Intune. The script runs through the ConfigMgr client, triggered from the Intune admin center.

Checkpoint

- ☐ Diagnostics zip of CL4 downloaded
- ☐ You can explain why CL2 has no Sync or Wipe

Lab 12.2 CL2 becomes cloud-only

25 min

Goal: Document CL2's current state, reset it and join it to Entra – without ConfigMgr and without Autopilot.

- 1 Record the old state: `dsregcmd /status` on CL2, the CL2 record in ConfigMgr, the Entra device object (join type, device ID) and the AD computer object.
- 2 Walk through the reset checklist from the slides and note what would be lost on a real device.
- On CL2: Settings → System → Recovery → Reset this PC → Remove everything.

- 4 While the reset runs, disconnect the network from the VM. When OOBE appears, reconnect the network and sign in as User2 with a work or school account. The disconnected network skips Autopilot – the VM may still be registered there from an earlier course.
- 5 Afterwards: `dsregcmd /status` and the new CL2 in Intune.

Checkpoint

- ☐ CL2: `AzureAdJoined` YES, `DomainJoined` NO
- ☐ CL2 enrolled in Intune, no ConfigMgr client
- ☐ The old records are noted for Lab 13.1

NOTE

Do not add CL2 to SCMI-ConfigMgr-Client – CL2 should end cloud-only.

Lab 13.1 Clean sweep

30 min

Goal: Remove ConfigMgr from CL4 and leave exactly one record of CL2 in every system.

- 1 **Stop the reinstall.** Intune admin center → Groups → SCMI-ConfigMgr-Client → Members: remove CL4. As long as CL4 is a member, the required `ccmsetup.msi` app installs the client again.
- 2 **Uninstall the client.** Intune admin center → Devices → Scripts and remediations → Platform scripts → Add → Windows: name SCMI-Uninstall-CMClient, upload a .ps1 file with the line below. Run this script using the logged-on credentials = No, enforce script signature check = No, run script in 64-bit PowerShell host = Yes. Assign it to SCMI-CL4-Group. A platform script runs once per device.

PowerShell

```
Start-Process -FilePath "$env:WINDIR\ccmsetup\ccmsetup.exe" -ArgumentList '/uninstall' -Wait
```

Checkpoint

- ☐ CL4: no CcmExec service, Intune still manages it
- ☐ Entra: exactly one CL2, join type Entra joined
- ☐ ConfigMgr: no CL2

OPTIONAL

Unlink the GPOs from `Cloud\Workstations\Sales`.

Lab 14.1 Break and fix

25 min

Goal: In pairs, one breaks and one diagnoses – using the five layers (identity, enrollment, authority, delivery, execution) and the log map only.

- 1 Pick a scenario and break it on your partner's pod while they look away.
- 2 Partner: write down the suspected layer before opening any log.
- 3 Diagnose and fix. Note the evidence that proved the cause.
- 4 Swap roles and pick another scenario. Time-box: 10 minutes per scenario.

Scenario	Symptom for the partner	How to break it
A · Authority	A new settings catalog profile does not apply on CL1	Device configuration slider to ConfigMgr
B · Detection	Firefox Win32 app reports Failed on CL4 although Firefox runs	Change the detection path of the Firefox app
C · Targeting	Notepad++ is assigned to CL4 but never shows up	Exclude filter on deviceName CL4 on the assignment
D · Signing	A ConfigMgr script detection fails again on CL1	Custom client setting, PowerShell execution policy All Signed, deployed to Pilot-CoMgmt

Checkpoint

- ☐ Two scenarios solved per person
- ☐ For each: layer, evidence, fix
- ☐ One runbook line you would give your helpdesk

Lab 15.1 Your migration plan

35 min

Goal: Draft the migration plan for your own environment in `SCMI_Migration_Plan_Template.xlsx`.

- 1 Assessment: devices, applications and deployment types, GPOs, identities.
- 2 Target state: join type per device population, authority per workload, what stays in ConfigMgr.
- 3 Waves: the first three waves with scope, rollback owner, go and rollback criteria.
- 4 Risks: top five with mitigation and owner.
- 5 Phases: exit criteria and target dates – including the clean sweep.
- 6 Prepare three minutes: target state, first wave, biggest risk.

Checkpoint

- ☐ A plan you can present in three minutes
- ☐ Every wave has a rollback owner
- ☐ The clean sweep has a date

Appendix

Deliberate dirt – do not fix

Object	Purpose
SmartETC Toolbox deployed to Dept-Sales	Detection points to the wrong folder – failed deployment visible via tenant attach
Adobe Reader 2019	Distributed, never deployed – dead content
7Zip	Duplicate application without deployment type
TEST_alt_Kopie, Neue Sammlung (2), Win10 Upgrade Wave 3 - DONE	Empty or meaningless collections
Finance Laptops, All Laptops	Empty on VMs – limiting chain and chassis query

Log map

Question	Where to look
Join state, PRT	<code>dsregcmd /status</code>
Hybrid join chain	Event Viewer → Microsoft → Windows → User Device Registration → Admin
MDM enrollment and policy errors	Event Viewer → DeviceManagement-Enterprise-Diagnostics-Provider → Admin
Co-management state	<code>CoManagementHandler.log</code> , <code>ConfigInfo</code> registry value
Cloud-first client registration	<code>ccmsetup.log</code> , <code>ClientIDManagerStartup.log</code> , <code>LocationServices.log</code>
Tenant attach upload and actions	<code>CMGatewaySyncUploadWorker.log</code> , <code>CMGatewayNotificationWorker.log</code> (site server)
Win32 apps from Intune	<code>IntuneManagementExtension.log</code> , <code>AppWorkload.log</code>
Platform scripts, remediations	<code>AgentExecutor.log</code> , <code>HealthScripts.log</code>
PSADT packages	<code>C:\Windows\Logs\Software</code>
ConfigMgr apps	<code>AppDiscovery.log</code> , <code>AppEnforce.log</code>
ConfigMgr baselines	<code>CIAgent.log</code> , <code>DcmWmiProvider.log</code>
Windows Update, Delivery Optimization	<code>Get-WindowsUpdateLog</code> , <code>Get-DeliveryOptimizationStatus</code>